

Product Problem Brief: RestoLaunch AI

1. The Context

Launching a physical restaurant or cloud kitchen requires massive upfront capital expenditure (CapEx) on real estate, kitchen infrastructure, and staffing. In hyper-competitive food capitals like Bangalore, market dynamics shift block-by-block, not just city-by-city.

2. The Core Problem Statement

Independent culinary entrepreneurs lack access to hyper-local, consolidated market intelligence, forcing them to make high-stakes capital allocation decisions based on intuition, anecdotal advice, or fragmented data.

3. The Root Causes

- **Information Asymmetry:** Platforms like Zomato hold the ground truth on consumer preferences (ratings, volumes, cuisines), but this data is optimized for consumers looking for a meal, not founders looking to open a business.
- **The "Micro-Market" Trap:** A cuisine format that is highly profitable in Indiranagar might completely bomb two kilometers away in Koramangala due to localized supply saturation or lower household spending thresholds.
- **Blind Spot Pricing:** Founders often price their menus based on their internal cost of goods sold (COGS) rather than the local neighborhood's optimal price-elasticity sweet spot.

4. The Business Impact

This empirical blind spot is the direct driver behind the industry's starkest metric: **60% of new restaurant concepts fail within their first 12 months**. This represents millions in lost capital, stranded inventory, and operational churn.

Technical Problem Brief: RestoLaunch AI System

1. User Input (The Request)

The system captures specific data points directly from the user (the aspiring restaurant owner) to define what kind of business they want to open. The LLM must capture these exact four variables:

- **Target Location:** The specific neighborhood they want to open in (e.g., "Indiranagar").
- **Operational Format:** The type of restaurant (e.g., "Cafe" or "Quick Bites").
- **Proposed Budget:** The average cost for two people (e.g., \$600\$).
- **Target Cuisine:** The type of food they want to sell (e.g., "Italian" or "North Indian").

2. Data Ingestion (Cleaning the Raw Data)

Load and preprocess the Zomato dataset from Hugging Face

(<https://huggingface.co/datasets/ManikaSaini/zomato-restaurant-recommendation>)

The system loads a static database of over 51,000 restaurants. Before doing any math, the system must clean the messy text data into strict numbers so the math does not break.

- **Fixing Prices:** Remove commas from string numbers (convert "1,200" to integer 1200).
- **Fixing Ratings:** Extract the first number from text like "4.1/5" to get the float 4.1. If a restaurant is marked as "NEW" or "-", assign it a safe average fallback number (like 3.5) so the code does not crash.
- **Fixing Votes:** Convert vote counts into simple integers. If there are no votes, set it to 0.

3. Integration Layer (Filtering the Noise)

Before running heavy calculations, the system must shrink the dataset to save computing power.

- **The Filter:** The system looks at the global 51,000+ restaurants and filters out everything that does not match the user's **Target Location** and **Operational Format**.
- **The Result:** This leaves a small, hyper-local list of about 500 to 800 direct competitors. All subsequent math is done *only* on this small list.

4. Recommendation Engine (The Scoring System Explained)

This is the core brain of the application. The goal is to find "Market Vacuums"—cuisines that local people love and order frequently, but that do not have enough physical restaurants supplying them. The LLM must execute these exact steps:

Step A: Calculate the Market Demand Score (MDS) for Every Restaurant

We cannot trust raw customer ratings alone. A restaurant with a \$5.0\$ rating from only 2 people is not actually better than a restaurant with a \$4.2\$ rating from 2,000 people. We must combine the rating with the volume of votes to find true popularity.

- **The Formula:**
$$\text{MDS} = \text{Cleaned Rating} \times \log_{10}(\text{Votes} + 1)$$
- **Why we use Logarithm (log10):** A logarithm shrinks massive numbers. This ensures a place with 10,000 votes doesn't completely break the scale compared to a place with 1,000 votes.
- **Why we add +1:** If a place has \$0\$ votes, the math $\log_{10}(0)$ causes a critical code error. Adding \$+1\$ ensures that zero votes safely equals a score of zero.

Step B: Calculate the Average Demand for the Target Cuisine

Now, the system groups all the local restaurants by the user's **Target Cuisine** (e.g., all Italian places in Indiranagar).

- It takes the MDS of every Italian restaurant in that neighborhood and calculates the average.
- **The Formula:**
$$\overline{\text{MDS}}_{\text{cuisine}} = \frac{\text{Sum of all MDS for that Cuisine}}{\text{Total number of restaurants serving that Cuisine}}$$
- **What this means:** This number represents exactly how hungry the neighborhood is for this specific food.

Step C: Count the Competitors (Supply Saturation)

The system simply counts how many restaurants are currently selling that specific cuisine in that neighborhood. Let's call this the **Competitor Count** (\$C\$).

Step D: Calculate the Final Opportunity Index (OI)

This is the final score. We want to find a high demand (high Average MDS) divided by low competition (low Competitor Count).

- **The Formula:**
$$\text{Opportunity Index} = \frac{\overline{\text{MDS}}_{\text{cuisine}}}{C + 1}$$
- **Why we add +1 to the bottom:** If there are exactly zero Italian restaurants in the area, dividing by zero crashes the code. By adding \$+1\$, the math works perfectly. More importantly, dividing a high demand score by \$1\$ keeps the score massive, which tells the system: *"This is a massive opportunity, tell the user to open this restaurant immediately!"*

5. Output Display (What the User Sees)

The backend sends the final calculations to the frontend screen. To prevent confusing the user with too much data, the display is kept incredibly simple:

- **The Verdict:** A simple text alert telling the user if their idea is a "High Opportunity" (high index score) or a "Red Ocean / Oversaturated" (low index score).
- **Top 5 Competitors:** The UI only displays a clean table of the top 5 direct competitors in their area, ranked highest to lowest by their individual MDS score.